

Тестовое задание для кандидата на должность СТО

Контекст: Платежная платформа для онлайн-школ (мерчантов) с интеграцией банковских рассрочек, эквайринга, BNPL и брокеров.

1) Единый партнерский API

Задача: Предложите интерфейс API для онлайн-школ:

- какие основные методы должны быть доступны;
- как реализовать версионирование и backward compatibility;
- как вы будете контролировать SLA и стабильность ответов внешних провайдеров

Ожидаемый формат:

- Таблица/список эндпоинтов с кратким описанием и примерами. Код писать не нужно.

Ответ:

1. Основные методы API (Единый партнерский API)

Цель API - абстрагировать мерчанта (онлайн-школу) от сложности внутренней платежной инфраструктуры (банки, BNPL, брокеры). Ключевые сущности: **Заказ** и **Платеж**.

Группа	Метод HTTP	Эндпоинт	Краткое описание	Пример использования online-школой
Сессии оплаты	POST	/v1/checkout-sessions	Создает сессию для оплаты. Ядро API. Возвращает ссылку на платежную форму и доступные способы оплаты.	Пользователь нажимает "Купить курс". Школа создает сессию и перенаправляет пользователя на платежную форму.
Управление платежами	GET	/v1/payments/{id}	Возвращает детализированный статус платежа	Проверка статуса платежа после возврата пользователя из платежного шлюза или для отображения в ЛК online-школы.

	POST	/v1/payments/{id}/cancel	Отмена платежа	Пользователь передумал покупать до завершения оплаты
	POST	/v1/payments/{id}/refund	Возврат средств (полный или частичный)	Ученик запросил возврат по договору. Школа инициирует возврат через API.
Управление заказами	POST	/v1/orders	Создает заказ (может быть отделен от платежа для сложных сценариев)	Создание предзаказа или корзины с несколькими курсами
	GET	/v1/orders/{id}	Получение информации о заказе и связанных платежах	Отслеживание истории покупок ученика
Подписки и счета	POST	/v1/subscriptions	Создание подписки с регулярными списаниями	Оформление доступа к курсу по ежемесячной подписке
	POST	/v1/invoices	Создание счета для оплаты (для юр.лиц или для ручного выставления)	Бухгалтерия online-школы выставляет счет корпоративному клиенту.
Вебхуки	POST	(Настраивается школой)	Механизм для асинхронных уведомлений о событиях	События: payment.succeeded, payment.failed, refund.processed. Online-школа получает мгновенное уведомление об успешной оплате и открывает доступ к курсу.
Справочники	GET	/v1/payment-methods	Список доступных способов оплаты (карта, BNPL, рассрочка) и их параметров (сроки, лимиты).	Отображение актуальных способов оплаты на сайте школы

2. Версионирование и Backward Compatibility

Принцип: Все изменения должны быть обратно совместимыми. Старая версия API должна работать до тех пор, пока все мерчанты не перейдут на новую.

Стратегия:

1. **Версионирование через URL:** [https://api.payment.ru/v1/...](https://api.payment.ru/v1/) — самый простой и прозрачный способ.
2. **Semantic Versioning для API: v1, v2.** Изменения:

- v1.1 -> v1.2: Добавление новых необязательных полей в ответ — обратно совместимо.
- v1.2 -> v2.0: Удаление или переименование обязательного поля — НЕ обратно совместимо.

3. Правила обеспечения Backward Compatibility:

- **Не удалять поля.** Помечать их как **deprecated** в документации и логике.
- **Не менять семантику существующих полей.** Тип и значение поля amount не должны меняться.
- **Добавлять только новые необязательные поля** в тело запроса.
- **Расширять enum'ы** (например новые статусы платежа), но не изменять существующие.

4. Жизненный цикл версии:

- При выпуске **v2** версия **v1** переходит в фазу поддержки. Новая функциональность добавляется только в **v2**.
- Публикуется clear timeline: «v1 будет поддерживаться до 01.01.2026. Перейдите на v2 до этой даты.»
- Активная коммуникация с мерчантами через email, changelog, и статус-страницу.

3. Контроль SLA и стабильности ответов внешних провайдеров

Это операционная и архитектурная задача, решаемая на нескольких уровнях.

1. Мониторинг и метрики

- **SLA-дашборды:** SLA доступности нашего API для мерчантов (напр., 99.9% uptime) и SLA доступности внешних провайдеров, от которых зависит наше API (банки-эквайеры, BNPL-сервисы, брокеры).
- **Ключевые метрики:**
 - **Доступность:** % успешных HTTP-запросов 5xx / total.
 - **Задержка:** P50, P95, P99 - время ответа нашего API и API, от которых есть зависимость.
 - **Рейтинг ошибок:** Детализация по типам ошибок (таймаут, 5xx от провайдера, бизнес-ошибки)
- **Оповещения:** Автоматические оповещения при росте ошибок в приоритетные чаты в мессенджерах.

2. Архитектурные паттерны для устойчивости

- **Автоматический выключатель (Circuit Breaker):**
 - Если провайдер BNPL начинает отвечать с 50% ошибок, «цепь разрывается». Наш API автоматически перестает слать провайдеру запросы на 30-60 сек., возвращая мерчанту ошибку «Способ оплаты временно не доступен».
- **Таймауты и логика восстановления:**
 - **Агрессивные таймауты** на вызовы провайдеров (например, 3-5 секунд)

- Восстановление только для идемпотентных операций (например, GET /payments/{id} и только для определенных ошибок (таймаут, 5xx))
- **Фолбэки (Fallbacks):**
 - При недоступности основного банка-провайдера автоматически переключаться на запасного
 - Если сервис рассрочек не отвечает, временно скрыть его из списка доступных методов оплаты

3. Операционное управление

- **Статус-страница:** Публичная страница с текущим состоянием API и интеграций (чтобы мерчанты видели, что проблема у конкретного провайдера, а не в нашей платформе)
- **Регулярные отчеты:** Ежемесячные отчеты для мерчантов по SLA и отчеты от нас для провайдеров с требованием улучшений.
- **Ясные договоренности:** В юридических договорах с провайдерами должны быть прописаны SLA и финансовые компенсации за их невыполнение.

2) Команда и процессы

Контекст:

Компания находится на этапе активного роста:

- Целевой результат:
 - Масштаб - 1 000+ подключенных онлайн-школ и 1 000 000+ транзакций/мес;
 - Оперативное подключение (срок 3 дня) новых брокеров и платежных инструментов;
 - Обеспечение соответствия 152-ФЗ и PCI DSS;
 - Платформа подготовлена к международному запуску (мультиязычность, мультивалютность, локализация).

Задача:

Опишите, какой командой (количество, состав, роли) и процессами разработки вы достигнете данной результата.

Ответ:

1. Команда (Target Headcount для масштаба 1000+ мерчантов)

Для поддержания и развития платформы с такими требованиями потребуется **~40-50 человек**, объединенных в продуктовые команды.

Роль	Количество	Ключевые обязанности и компетенции
Технический руководитель (CTO/Head of Tech)	1	Стратегия технологии, архитектура, управление командой, бюджетирование, соответствие стандартам, партнерство с провайдерами.
Продуктовый менеджер (CPO/Head of Product)	1	Формирование продуктового видения, Roadmap, приоритизация, составление и анализ продуктовых метрик.
Team Lead / Engineering Manager	3-4 (по 1 на команду)	Операционное управление инженерами, мониторинг, решение технических blockers
Backend Developer (Core platform)	8-10	Разработка и поддержка ядра платежной системы, API, интеграции с провайдерами. Языки: Go, Java, Python
Backend Developer (Compliance & Data)	4-5	Обеспечение 152-ФЗ, PCI DSS, безопасность, шифрование, работа с персональными данными.
DevOps / Platform Engineer	4-5	Инфраструктура (K8s, Terraform), CI/CD, мониторинг (Prometheus, Grafana), обеспечение отказоустойчивости и SLA
QA Engineer (Manual & Auto)	4-5	Тестирование API, интеграционное тестирование, нагрузочное тестирование, автоматизация

		регрессионных тестов.
Frontend Developer	3-4	Разработка платежного виджета, панели управления для мерчантов, внутренней админ-панели.
Data Engineer / Analyst	2-3	Построение ETL-пайплайнов, дэшборды для бизнес-аналитики, фрод-мониторинг.
SRE / Инженер поддержки	4-5	Мониторинг провайдеров, инцидент-менеджмент, поддержка мерчантов L2/L3.
Системный архитектор	1-2	Проектирование высоконагруженной и отказоустойчивой архитектуры.
Security Officer	1-2	Ответственный за PCI DSS и 152-ФЗ, проведение аудитов, разработка политик безопасности.
Технический писатель	1	Документация API, онбординг мерчантов, внутренняя база знаний.

Организационная структура

Команды, сфокусированные на потоках создания ценности.

- **Команда «Ядро и API»:** Ответственность за стабильность основного API, производительность, backward compatibility.
- **Команда «Интеграции и экспансия»:** Ответственность за быстрое подключение новых провайдеров (брокеров, BNPL), международную локализацию.
- **Команда «Безопасность и Соответствие»:** PCI DSS, 152-ФЗ, фрод-мониторинг, безопасность данных.
- **Команда «Инфраструктура»:** Ответственность за предоставление внутренней инфраструктуры для разработки: CI/CD, мониторинг, инфраструктура как код.

2. Процессы разработки и операционной деятельности

2.1. Процесс разработки: Гибридная модель (Scrum + Канбан)

- **Для продуктовых фич: Scrum** с двухнедельными спринтами (Позволяет планировать и стабильно доставлять новый функционал)
- **Для интеграций и баг-фиксов: Kanban.** Позволяет быстро реагировать на запросы от партнеров и инциденты, минимизируя цикл времени обработки.

Ключевые практики:

- **Trunk-Based Development:** Короткоживущие feature-ветки (макс 1-2 дня), частые мержи в main, непрерывная интеграция.
- **Feature-flags:** Все новые значимые фичи, особенно интеграции, выкатываются под флагом, таким образом позволяя включать функционал для определенных мерчантов или тестировать в продакшене с минимальными рисками.
- **Автоматизированное тестирование:** Многоуровневая пирамида: Юнит-тесты -> Интеграционные тесты (на моках провайдеров) -> Контрактное тестирование (Pact) -> Выборочное E2E.
- **CI/CD:** Полная автоматизация сборки, тестирования и развертывания. Развертывание в продакшен — несколько раз в день.

2.2. Процесс операционной деятельности и надежности (SRE)

■ Мониторинг и нотификация

- 2.2.■.1. **Внешний:** Synthetic-мониторы для ключевых сценариев API (создание сессии, проведение платежа)
- 2.2.■.2. **Внутренний:** Детальный мониторинг всех интеграций с провайдерами (латентность, ошибки 5xx, таймауты)
- 2.2.■.3. **SLA-дэшборды:** Публичные и внутренние дэшборды в Grafana с расчетом доступности по процентилям (P95, P99)

■ Управление инцидентами

- 2.2.■.1. Четкий он-кол роутер для SRE/дежурных инженеров.
 - 2.2.■.2. Ретроспективный анализ по каждому серьезному инциденту с фиксацией выводов во внутреннем и опционально в публичном информационных полях (внутренняя публичная информационная система - вики, конфлюэнс и т.д., публичная - статус API)
- Управление ёмкостью
- 2.2.■.1. Регулярное нагрузочное тестирование для определения узких мест перед пиковыми нагрузками (например, начало учебного сезона)

- 2.2.■.2. Прогнозирование роста и автоматическое масштабирование инфраструктуры

2.3. Процесс обеспечения безопасности и соответствия

■ PCI DSS:

- 2.3.■.1. Выделенный сегмент сети (CDE - Cardholder Data Enviroment)
- 2.3.■.2. Регулярное сканирование уязвимостей (ASV)
- 2.3.■.3. **Токенизация карточных данных.** Сами карточные данные никогда не попадают на наши сервера, используем токены от PCI DSS-совместимого провайдера.

■ 152-ФЗ:

- 2.3.■.1. Шифрование персональных данных при хранении и передаче.
- 2.3.■.2. Система разграничения прав доступа (RBAC).
- 2.3.■.3. Ведение журналов учета доступа к данным.
- 2.3.■.4. Назначение ответственного за соблюдение закона.

2.4. Процесс быстрого подключения провайдеров (3 дня)

Для достижения цели в 3 дня необходим максимально стандартизированный и автоматизированный процесс.

1. Этап 1: Подготовка (1 день). Юридические договоренности, обмен ключами API, доступ к тестовой среде провайдера.
2. Этап 2: Разработка (1 день). Благодаря унифицированному внутреннему API для всех типов провайдеров, разработчик не пишет логику с нуля.
 - Создается адаптер для нового провайдера, который реализует стандартный интерфейс (например, `PaymentProviderInterface` с методами `createPayment()`, `getStatus()`, `refund()`)
 - Интеграция помечается feature flag.
3. Этап 3: тестирование и выпуск (1 день)
 - **Автоматические тесты:** набор стандартных новых тестов прогоняется против нового адаптера.
 - **Контрактное тестирование:** Проверка, что провайдер соблюдает заявленные в документации контракты.
 - **Выпуск:** Включение функционала для всех мерчантов или только для пилотных под feature flag.

Ключевой технологический элемент: Внутренняя `Abstract Provider Gateway`, которая абстрагирует ядро системы от специфики каждого провайдера. Это позволяет подключать новых партнеров как плагины.

Резюме: Как это работает вместе

- **Масштаб 1000+ школ:** Достигается за счет надежной, отказоустойчивой архитектуры (команда SRE/DevOps) и качественного API (команда Ядра).
- **Подключение за 3 дня:** достигается за счет **стандартизированного процесса, feature flags** и **абстрактного шлюза провайдеров** (команда Интеграций).
- **Соответствие стандартам:** Достигается за счет **выделенной команды и встроенных в процесс практик безопасности** (команда Безопасности).
- **Международный запуск:** Достигается за счет **архитектуры, позволяющей оперативно подключать новые страны** и команды Интеграций, которая быстро добавляет новые локалы и валюты.

Такой подход позволяет масштабироваться быстро, но контролируемо, сохраняя высокий уровень надежности и безопасности.

3) R&D и искусственный интеллект

Приведите 3 идеи, как можно использовать **AI/ML** или **аналитику** для:

- скоринга заявок и антифрода
 - Поведенческий антифрод-скоринг в реальном времени (ML-модель анализирует не только статические данные, но и динамику поведения пользователя во время сессии оплаты, чтобы выявить аномалии)
 - автоматизации поддержки онлайн-школ.
 - AI-помощник для анализа инцидентов на основе данных о платежах
 - Проактивная поддержка и Health Score для мерчантов (анализ метрик каждой школы и выработка персональных рекомендаций по увеличению продаж)
-

Формат представления результата

- Ссылка на документ + экспорт в PDF.
-